

Systematic Generation of Dynamic Occlusion-Caused Collisions

Eli-Henry Dykhne, Shea Anthony Cardozo, James Riddell, Sean Sedwards, Krzysztof Czarnecki

Abstract—Dynamic occlusions (occlusions caused by other moving road objects) pose some of the most difficult challenges for autonomous driving systems (ADS). Validating the robustness of ADSs to safety critical dynamic occlusions is a difficult task due to the rarity of such scenarios in recorded driving logs. We provide a systematic typology of dynamic occlusion scenarios involving vehicles, as well as an interpretable framework for ADS safety validation in the presence of dynamic occlusions. Our framework allows for the generation of a diverse set of dynamic occlusion-caused collisions (OCCs) across a wide variety of intersections. These OCCs are tailored directly for the ADS under test. We have implemented our method of generation in a tool (OCC-Gen) and provide results demonstrating that it achieves higher generation efficiency and diversity of OCCs than prior works, while being applicable to any map or intersection. Our tool allows for the custom tailoring of scenario generation, occlusion calculation, and behaviour of non-ego vehicles, as well as driver reaction time. In this work, we present our technique and provide a detailed analysis of the variety and quality of our generated scenarios.

Index Terms—Dynamic occlusions, autonomous driving, ADS, scenario generation, validation.

I. INTRODUCTION

THE difficulty of collecting and testing sufficient quantities of dangerous and unusual road scenarios remains a limiting factor in validating autonomous driving systems. Prior work has mostly focused on testing the ADS through simulated replay of real-world scenarios from on-road datasets captured by data collection vehicles [3, 20]. This has led to significant research in critical scenario generation, where the behaviour of ADSs can be verified against adversarial agents [26]. However, comparatively little research has focused on generating critical scenarios caused by occlusions. Even less attention has been given to the generation of *dynamic occlusions*, which are responsible for a significant portion of collisions [22]. The scarcity of scenarios is even more acute in the area of dangerous dynamic occlusions, because while dynamic occlusions are not rare on the road, scenarios where they cause collisions are. The potential variety of such scenarios also poses a problem for validation, as any scenario where three vehicles line up while two of them are on intersecting paths could potentially result in an occlusion-caused collision. Prior works have focused on particular types of OCCs [1, 13, 21, 30, 31] but make no attempt to capture the complete range of possible scenarios. The generation of dangerous occlusion scenarios for the validation and training augmentation of ADSs across a variety of roadways is vital to ensuring the safety of such systems.

In this work, we present OCC-Gen. Our approach to generate these dangerous scenarios, applied to the nuPlan dataset [3]

which itself contains no such scenarios. We introduce an extensive typology of dynamic occlusion scenarios and produce these scenarios in significant quantities, encompassing diverse road geometries and reactive drivers. Furthermore, we achieve this while preserving a high level of diversity within our defined typology.

II. DEFINING DYNAMIC OCCLUSIONS

A general traffic occlusion occurs when an object obstructs a road user from seeing another part of the road. This prevents the road user from seeing a potential threat, such as a traffic conflict or damage to the road. A traffic conflict is a situation in which two or more road users maintain trajectories such that there is a risk of collision [16].

A. Static VS Dynamic Occlusions

Whether an occlusion is static or dynamic depends on the nature of the occluder. Static occlusions are caused by objects that are stationary and not expected to move soon, such as trees, buildings, and parked cars. Dynamic occlusions are primarily caused by moving vehicles, or those that may move shortly. Trucks, buses, cars can occlude other cars, vulnerable road users (VRUs), and foreign object debris (FOD).

While static occlusions are localized to the blind spots generated by buildings and large obstructions, dynamic occlusions can occur anywhere on the road, making identifying and dealing with them a much harder task. For this work, we focus on dynamic occlusions where all three objects are vehicles.

B. Dynamic Occlusion Typology

The type of dynamic occlusion is dependent on the traffic conflict type, which stems from the relative positioning of the two conflict vehicles [5, 6, 15], as well as the positioning of the occluding vehicle relative to the two conflict vehicles. A conflict vehicle is defined as a vehicle on a collision course with ego (the focus vehicle). Traffic conflicts between vehicles occur at conflict points. Static conflict points are the results of road geometries, and occur where lanes cross, merge and diverge. Dynamic conflict points are not tied to static road geometries but instead occur due to the movements of the agents on the road, such as lane changes, jaywalking or differences in the speed of vehicles sharing the same lane.

Table I illustrates dynamic occlusion patterns with respect to combinations of conflict types and types of occluding vehicle.

Vehicle to Vehicle Conflict Types: Vehicle-to-vehicle conflicts reduce to the five base conflict types: Sequential, Cross,

TABLE I
ILLUSTRATIVE EXAMPLES OF DYNAMIC OCCLUSION PATTERNS

Conflict Occluder	Sequential	Cross	Merge	Diverge	Head-On
Sequential	N/A				N/A
Cross					
Merge					
Diverge					
Parallel					

Arrows indicate conflict vehicle movements. Circle denotes occluder, bar denotes vehicle is stopped.

Merge, Diverge, and Head-On [5, 19]. In this work, we exclude head-on conflicts due to the absence of two-way roads in the nuPlan dataset, although our methods are adaptable to maps containing such specialized lanes. Vehicle speeds, lane structures, and traffic control devices such as traffic lights significantly impact the likelihood and severity of these conflicts. In order to inoculate an ADS system against dynamic occlusions, it is important to cover as many types, on as large a variety of intersections as possible.

Movement of Occluding Vehicle: The positioning and movement of the occluding vehicle is crucial in determining the type of dynamic occlusion. Occluding vehicles can be either Sequential, Cross, Merge, Diverge, and Parallel with respect to the conflict vehicles. Depending on the geometry of the intersection, a single scenario may feature an occluder that fits multiple categories simultaneously, such as an occluder crossing the path of one agent while being parallel to another. In such cases, the scenario is classified as both types.

The full type of a given occlusion scenario consists of two parts. First, a label for the relative geometry of the lanes used by the two conflict vehicles. Second, a label for the geometry of the occluder relative to the two conflicting vehicles. For example, a typical cut-out scenario can be classified as a sequential conflict with a diverging occluder. As more agents become part of the scenarios, more complex occlusions can be created, where multiple occluders work together to create OCCs.

III. METHOD

Our method broadly follows the technique outlined in [13], where real scenarios involving vehicles, pedestrians and obstacles are captured by either external or onboard sensors and then injected with dynamic occlusions, with the potential to cause an OCC, by injecting extra vehicles into the scene. Unlike [13], where only the occluder is injected, we also inject the conflict vehicle in order to increase the likelihood

of generating a collision. Also, unlike [13], the injection of occluder and conflict vehicles is guided by our systematic occlusion typology. The generated scenarios are then run with occlusions enabled to see which result in a collision. This produces a subset of scenarios where collisions occur, but where the collision may or may not be caused by the presence of the occlusion. For example, the vehicles might have already been on an unavoidable collision course. This subset of scenarios is then re-run with occlusions turned off, as if all vehicles are completely see-through. Only those scenarios where the collision does not occur in the see-through regime are classified as OCCs.

A. NuPlan

Our implementation makes use of the nuPlan simulator [3]. The nuPlan simulator is a lightweight, extensible platform for evaluating motion planners. It can operate in both open and closed loop modes, while tracking planning-specific metrics. In addition to adapting the simulator to our needs, our work makes use of nuPlan’s extensively labelled dataset of on-road scenarios, collected by driving a vehicle with multiple cameras and LiDAR sensors for 1500 hours across Las Vegas, Pittsburgh, Boston, and Singapore.

B. OCC-Gen

OCC-Gen is a modified version of the nuPlan simulator [3]. The tool is capable of selecting planners for the ego agent, and a separate type of planner for the other agents in the closed loop reactive regime.

We allow background agents to be controlled by Urban_Planner [29], IDM [3], GC_PGP [11], and the three variations of the PDM planner from tuPlan [7]. Any planner can be integrated to control ego.

OCC-Gen includes our custom occlusion system, which can be extended with custom occlusion modules. It also

implements our custom notice system. Our tool provides a method for exporting, loading and rerunning scenarios in custom nuPlan scenario collection files (NPSC). This allows the same scenarios to be run again with different settings.

C. Agent Control

For our experiments, we control the ego agent with a motion planner. Recorded pedestrians, cyclists, and non-vehicle objects are simply controlled via playback. Non-ego vehicles are controlled by a separately chosen vehicle planner, except for vehicles determined to be parked, which remain stationary. Vehicles that are both far from the ego vehicle and on non-intersecting paths, are controlled by a simple IDM controller, to reduce unnecessary computation.

In our experiments, for both the ego vehicle and all other vehicles requiring a planner, we chose the winning entry of the 2023 nuPlan challenge [3]. Specifically, we selected the PDM-Closed model [7], which performed best in the closed reactive portion of the nuPlan challenge. The PDM-Closed model is based on simple center-line following with positional and speed offsets, while simulating forward via an internal model of the nuPlan simulation environment. The trajectory that produces the highest scoring result from the heuristic metrics is then chosen.

This model was chosen to maximize the likelihood of safe driving and collision avoidance, as well as to benefit from its faster inference time compared to many heavier neural network-based alternatives. Although we selected this particular model to control our drivers, our method and tooling are capable of optimizing collisions for different planners. Ego and other drivers do not need to use the same planner.

All planners are given access to the HD map in whatever format they require (raster, vector, etc.), as well as the history of their own prior trajectory and the dimensions and trajectories of all other tracked objects. The visibility of other tracked objects in this history is determined by our occlusion computation and the notice system, which determines if the vehicle has had enough time to notice the other object.

D. Occlusion Computation

In order to determine if objects were visible or occluded, we implement the occlusion computation algorithm as described in Alg. 1 and visualized in Fig. 1. The occlusion computation treats objects, pedestrians, and cyclists as see-through while vehicles are opaque, and can thus cause other objects to become occluded. In order to increase the speed of computation, we use a lightweight proxy for ray casting by creating 720 triangular wedges in a disc with ego as the centre-point to emulate its field of view. This number results in an angular resolution of 0.5° , which is in the range typical for LiDAR sensors used in autonomous driving [27]. For a vehicle to be visible, it must be intersected by at least 3 wedges. This allows the algorithm to simulate small pieces of cars being exposed without making the whole vehicle appear, much like a human or object detection algorithm would be likely to miss an object if only a sliver was exposed from behind another object. The algorithm works by processing the objects from nearest to

furthest. It checks that the wedges that are in the sector that should cover the current object are still present, and then if the object is marked as opaque, it removes all wedges intersecting that object, such that all future checks against objects further away will be unable to check in that sector which is being occluded by this current object. This allows us to reduce the number of checks that need to be made in the future. As noted in [13], the occlusion relation is asymmetric. It is possible for one vehicle to be occluded from ego's view, while it still being capable of seeing ego, and vice versa.

Algorithm 1: Determine Occlusions

```

input:   observer: Object we calculate occlusions for
           targets:   Objects which may be occluded
           numWedges: Controls angular resolution
           minHits:   Minimum wedge hits to be visible

1 wedges  $\leftarrow$  GenWedgePolys(numWedges)
2 sortByDistance(targets, observer)
3 notOccluded  $\leftarrow$   $\emptyset$ 
4 for target  $\in$  targets do
5   wedgeHits  $\leftarrow$  0
6   targetCorners  $\leftarrow$ 
     ConvertToObsFrame(observer, target)
7   targetPoly  $\leftarrow$  Polygon(targetCorners)
8   angle  $\leftarrow$  GetAngleToTarget(observer, target)
9   ctrWedgeIdx  $\leftarrow$  int(angle/ $2\pi$   $\times$  numWedges)
10  toRemove  $\leftarrow$   $\emptyset$ 
11  diameter  $\leftarrow$ 
     AngularDiamObsPOV(target, observer)
12  numWedgesToCheck  $\leftarrow$ 
     LengthToNWedges(diameter, numWedges)
13  for i  $\leftarrow$   $-\text{numWedgesToCheck}/2$  to
     numWedgesToCheck}/2 do
14    wedgeIdx  $\leftarrow$  (ctrWedgeIdx + i)
     mod numWedges
15    if wedgeIdx  $\in$  wedges then
16      wedge  $\leftarrow$  wedges[wedgeIdx]
17      if wedge.intersects(targetPoly) then
18        wedgeHits  $\leftarrow$  wedgeHits + 1
19        if target is a VEHICLE then
20          toRemove  $\leftarrow$  toRemove  $\cup$  wedgeIdx
21  if wedgeHits  $\geq$  minHit then
22    notOccluded  $\leftarrow$  notOccluded  $\cup$  target.trackToken
23  for key  $\in$  toRemove do
24    delete(wedges[key])
25 return notOccluded

```

E. Reaction Time, Evidence Accumulation, and Attentiveness

Neither humans nor machines can instantly react to changes in the driving environment. Humans are limited by the time it takes to process new information and then make physical braking and steering inputs. ADSs are limited by a similar processing lag, as well as the time it takes to determine if a

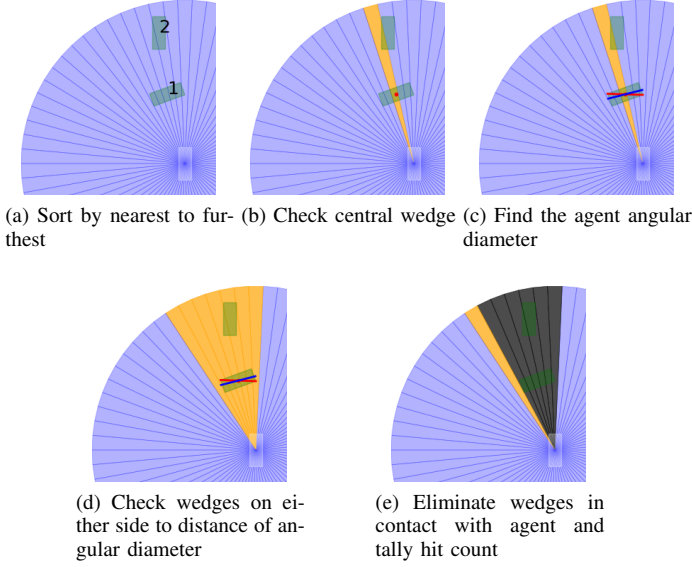


Fig. 1. The steps taken to detect which objects are occluded. (a) The agents are processed nearest to furthest. (b) The wedge pointing to the centre of the current agent is found. (c) Once the angular diameter of the agent is found, (d) the number of wedges to check to either side of the wedge pointing to the centre of the agent is determined. We continue with the next step even if the initial wedge is no longer in our set. If enough wedges intersect the agent, it is determined to be visible. (e) Wedges that intersect our agent are removed from the overall set of wedges, meaning that objects in the resultant shadow will be occluded.

given detection is a false positive or not. This is often done by requiring objects to be detected for multiple frames before they are confirmed as real objects and sent to the planning system to be added to its current world-view.

Researchers de Winter et al. [8] found that humans can typically react and apply brakes within 0.3 to 1.0 s, to scenes that have artificial occlusions introduced by blanking the screen of a simulator. The test subjects in this experiment were given practice runs and asked not to make the decision to brake until after the occlusion lifted, which is likely to have primed them and improved their results. Experimental research in cognitive neuroscience shows that humans accumulate evidence, such as visual stimuli, up to some threshold, before recognizing objects and making decisions [25]. Evidence accumulation has been linked to perception-reaction time in driving [10, 18], which is influenced by the accumulation rate (e.g., accumulating sightings of a highly occluded vs. fully visible object), surprise level (whether evidence matches prior expectations), and distraction level. In [2, 9, 17], drivers were measured as having reaction times between 0.5 and 1.7 s. A significant increase was found in those that compared phone distracted driving with undistracted driving. Drivers reacting to a looming vehicle (one that has braked unexpectedly while remaining in full view) [23] were observed with comparable reaction times centering on 1.04 s.

To model evidence accumulation and realistic reaction times, we implement the *notice system*, illustrated in Fig. 2. In order for an object to be visible to a planner at a given time-step in the current observation history buffer (OHB) of its trajectory, it must be both visible (as determined by the

t=6	1	2	3	4	5	6	7	8
Visible	×	×	×	✓	×	✓		
Noticed	×	×	×	×	×	×		
OHB	×	×	×	×	×	×		

t=7	1	2	3	4	5	6	7	8
Visible	×	×	×	✓	×	✓	✓	
Noticed	×	×	×	×	×	×	×	
OHB	×	×	×	×	×	×	×	

t=8	1	2	3	4	5	6	7	8
Visible	×	×	×	✓	×	✓	✓	×
Noticed	×	×	×	✓	✓	✓	✓	✓
OHB	×	×	×	✓	×	✓	✓	×

Fig. 2. The view of the present and past that the motion planner is given of this object. OHB refers to “Observation History Buffer”, provided to the planner. The shaded blue region visually displays the uncloak window. This example demonstrates the notice system with a 5-frame uncloak window and a 3-frame notice threshold. The object only becomes visible in the observation history at time-step 8, meeting both conditions of 3 frames of visibility and being visible at the start of the notice window. This shows that an object can be noticed while invisible (e.g., step 5 during time-step 8) and can be noticed on an invisible step (e.g., time-step 8). Previously unnoticed time-steps can become noticed once the system processes the appearance, revealing the trajectory for all steps that are both visible and noticed.

aforementioned occlusion detection algorithm), and “noticed”. We implement the notice system by using two tunable values that we call the *uncloak window* (tuned to be 1.5 s) and the *notice threshold* (tuned to be 1.0 s). To be noticed, an object must satisfy two conditions. First, it must have been visible, or already noticed at the beginning of the uncloak window, which ends at the present time. This condition simulates the reaction time of the system from the first sight of the object to it being noticed and reacted to, which is equal to the uncloak window duration. Second, within the uncloak window, the object must be visible for at least the duration of the notice threshold (though not necessarily contiguously) in order to stop vehicles behind imperfect occlusions that only flicker into view for a single frame, from being detected. This simulates evidence accumulation and how vehicles only visible for a short period of time can be missed by humans, or classed as false positives by ADSs. The uncloak window must be greater than or equal to the notice threshold. In this way, it is possible for objects at older time-steps, which were not visible before, to become visible at later time-steps.

F. Scenario Generation Techniques

Our techniques for generating scenarios from the conflict taxonomy classes all rely on the following basic steps. We select a scenario from the dataset, insert a vehicle on a trajectory that will be in conflict with the ego vehicle, and then insert another vehicle in such a position that it occludes the conflict vehicle. The specific insertion strategies vary based on the type of OCC we are trying to generate. While our tools allow generators for any type of scenario to be created, our work has focused on creating a separate generator for each type of conflict vehicle position, including Sequential,

Cross, Merge, and Diverge. Our OCCs are all generated on intersections, or areas where lanes cross, merge, or diverge from one another. While both the conflict vehicle and the occluder is inserted, care is taken to insert them into plausible locations, with plausible speeds and goals.

1) *Cross and Merge Scenarios*: The construction of the cross-type scenarios and the merge-type scenarios only differ in how the conflict point is selected. The cross scenario generator ensures that only lane connectors that cross the planned path of the ego vehicle are chosen as potential conflicts, while the merge generator only selects lane connectors that are merging with ego's planned path (excluding connectors that are currently on red lights). The general steps of our cross scenario generator are outlined below, as well as in Fig. 3.

- 1) We begin by loading in a real life scenario like Fig. 3a. These scenarios are initially chosen by selecting only those that involve ego traversing an intersection.
- 2) We optionally remove all vehicles that are not ego, shown in Fig. 3b. Keeping other vehicles in the scene can sometimes give us more occluded areas, allowing for more varied occlusion scenarios, while removing them increases the chances of generating exactly the type of OCC that this generator is designed for (cross/merge).
- 3) We select lane connectors that cross the lane connectors on ego's immediate path, and identify the conflict point for each one as shown in Fig. 3c.
- 4) For each conflict point, we generate candidate conflict vehicles at multiple different speeds. The speeds are chosen via a handful of offsets which are applied to the speed of the ego vehicle. Each conflict vehicle is positioned on the conflicting lane connector such that at its chosen speed, it will reach the conflict point at the same time as ego as shown in Fig. 3d. In order to filter out poor spawn locations, the vehicles are simulated forwards for a short period of time to ensure that they do not collide with ego prematurely. If the newly inserted vehicle is already occluded by other vehicles, then the process is done, and we call such a scenario a "natural occlusion". Otherwise, we move on to the next step.
- 5) To determine potential occlusion insertion locations, we create an FOV polygon from the centre of the ego vehicle to the left and rightmost edges of the conflict vehicle we want to obscure, as shown in Fig. 3e. Obstacles on the road, including road edges and other cars are subtracted from the FOV polygon in order to remove areas that could result in invalid spawn points.
- 6) To determine the candidate locations to spawn our occluder, we then take an intersection of the centre-lines of the lane connectors and the remaining FOV polygon as shown in Fig. 3f.
- 7) The resulting lines remaining from the intersection are discretized into points, which then make up our candidates as shown in Fig. 3g.
- 8) For each candidate point, we attempt to insert an occluder as shown in Fig. 3h. To avoid generating large amounts of scenarios with similar occluders, a minimum distance between accepted insertion points is enforced. Inserted vehicles are aligned with the direction of travel

of their lane connector.

- 9) We create a polygon to catch other vehicles in the original scene. This polygon includes vehicles in the same lane connector, as well as outgoing lane connectors. All vehicles in this polygon (even if we removed them in the first step) are collected and sorted from nearest to furthest. Vehicles in adjacent lanes that are within a smaller range are also included, excepting those observing red lights. The occluder is assigned the speed of the nearest vehicle in this set. The goal is set the same way. If a valid vehicle does not exist, the speed limit is selected, and a random goal is given.
- 10) Having injected a potential occluder, we perform a check that the insertion actually does occlude our conflict vehicle. We repeat the premature collision check. If all checks pass, we submit the new scenario to the set of generated scenarios for testing.

2) *Sequential Scenarios*: The construction of sequential scenarios follows a similar pattern. The key differences lie in the injection of the conflict vehicle. Unlike the cross scenarios, all sequential scenarios require the inserted conflict vehicle to be on the same lane as the ego vehicle, which severely reduces the number of possible scenarios that can be generated. These vehicles are inserted ahead of the ego vehicle, such that they are a fixed time gap ahead, and have their goals copied directly from ego. Their initial speed is set to 0, to simulate the conflict vehicle being unexpectedly stopped in the intersection, such as if they had to yield to a pedestrian or another vehicle. The process for inserting an occluding vehicle remains the same, except for the goal having to be selected so that the occluder takes a different route than ego, such that it can get out of the way and allow ego to have a chance to collide with the conflict vehicle.

3) *Diverge Scenarios*: The diverge generator targets diverging conflict vehicles and diverging occluders only, i.e., (diverge, diverge) cell of Table I, as we found the other cells in the diverge column difficult to instantiate in the used maps. Diverge scenarios share similarities with sequential scenarios. Key differences include the fact that the conflict vehicle diverges from ego, while the occluder diverges in parallel ahead or behind the conflict vehicle or mutually to it. A stopped conflict vehicle is inserted just ahead of the conflict point. For example, the scenario shown in (diverge, diverge) cell of Table I shows the conflict vehicle diverging in parallel with and ahead of the occluder.

IV. EXPERIMENTS AND RESULTS

Here, we explore the efficiency of our technique, as well as its ability to generate diverse OCCs. We determine the severity of the OCCs generated to show how vital such testing is and observe what mistakes even competition winning planners can make as well as their consequences. Finally, demonstrate the robustness of our method to faster reaction times, to show that even a responsive system must take occlusions into account in order to guarantee the safety of everyone on the road.

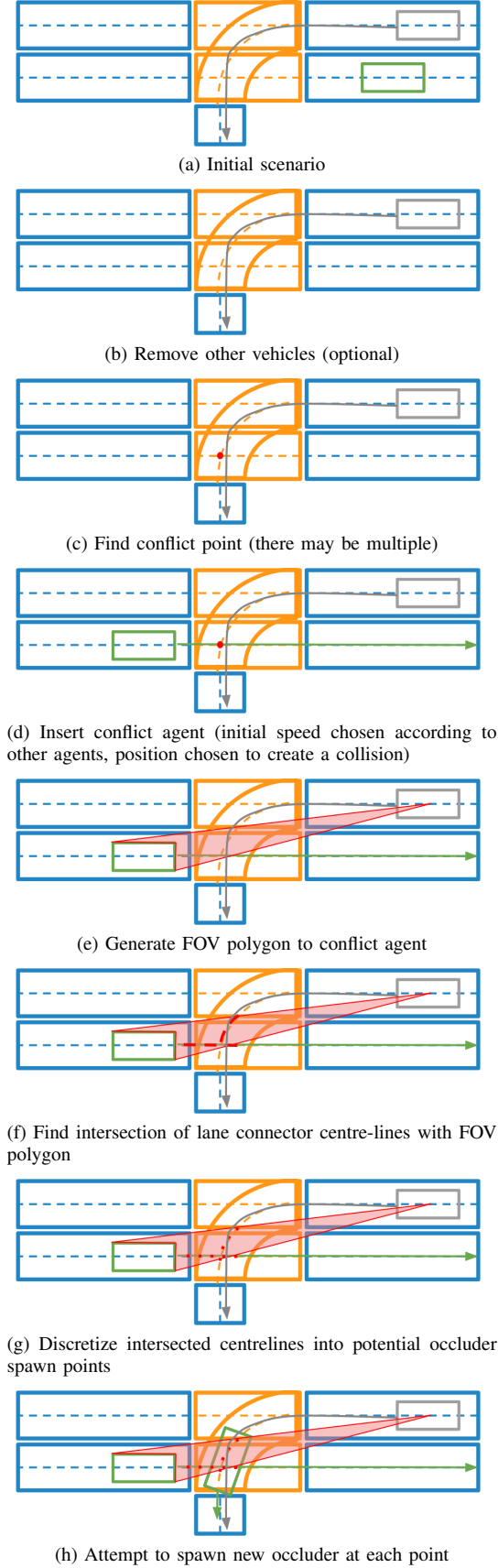


Fig. 3. The steps taken to generate an OCC using the cross conflict type generator. Ego is grey, other vehicles are green. Lanes are in blue, lane connectors are red. Planned paths are denoted by arrows.

A. Lone Occluder Insertion Ablation

Previous works [13] have operated by inserting only occluders into the scene. In order to compare this method of generation to ours, a pared-down version of our cross-conflict generator was created to see how many scenarios it could generate while selecting a potentially dangerous vehicle to ego, and then inserting an occluder that would block the view to ego. The number of OCCs generated could then be compared to the number of OCCs generated by our cross generator. Unfortunately, while this generator did produce candidate scenarios, it did not produce a single OCC, largely because having multiple vehicles arranged in such a way that they would collide if they continued onward is relatively rare without setting it up in each scene deliberately. Some scenes that are pulled from the dataset are also fairly sparse, without any vehicles close enough to be dangerous near the beginning of the scenario. Human drivers learn to avoid scenarios that risk resulting in a collision, and so few examples in the recorded traffic position themselves in a way that an inserted occluder can take advantage of. A more thorough search would be required to generate a few OCCs. It is therefore understandable why methods like those described in [13] required such a high number of generated scenarios to generate a similar number of OCCs as our own methods.

B. Typology Coverage

Our generation techniques achieve a sizable coverage of the possible OCC types described in our typology. Table II displays the full coverage we were able to achieve across our generation testing, while Table III, as a reference, displays the specific coverage of the configurations possible on a standard signalized two-lane cross intersection, that is, one with a single lane in each direction on each of the four intersection legs. Columns describe the relative motion of the conflicting vehicles, and rows describe the motion of the occluder relative to the conflicting vehicles. Cells marked with N/A denote configurations that are judged by the authors to not be geometrically feasible. We achieve a large, but not complete coverage over all types of OCCs thanks to our strategy of using four different generators, each one tuned to produce a single class. We achieve even better coverage when we limit the possibilities to those that can be feasibly achieved on the reference intersection. Note that the nuPlan dataset contains complex multi-lane intersections, which allow covering additional OCC types compared to the reference intersection.

Our weakness in generating scenarios where the occluder merges or is parallel is likely due to the small number of scenarios that occur on intersections with two parallel lanes in all directions, where a parallel or merging vehicle can obstruct ego's view while still leaving a path open for the collision to occur between ego and the conflict vehicle. Our difficulty in generating OCCs where the conflict vehicle diverges is down to the geometric difficulty of creating a diverge conflict in the first place. Such a conflict relies on abnormal curving roadways before the intersection that are

TABLE II
GENERATION COVERAGE OF THE TYPOLOGY

	Sequential	Merge	Diverge	Cross
Sequential	N/A	✓	✗	✓
Merge	✗	✗	✗	✓
Diverge	✓	✓	✓	✓
Cross	✓	✓	✗	✓
Parallel	✗	✗	✗	✓
Coverage	2/4	3/5	1/5	5/5

TABLE III
GENERATION COVERAGE OF THE TYPOLOGY ON A SIGNALIZED
TWO-LANE CROSS INTERSECTION

	Sequential	Merge	Diverge	Cross
Sequential	N/A	✓	✗	✓
Merge	✗	✗	N/A	N/A
Diverge	✓	✓	✓	✓
Cross	✓	✓	✗	✓
Parallel	N/A	N/A	N/A	N/A
Coverage	2/3	3/4	1/3	3/3

TABLE IV
UNIQUE INTERSECTION GENERATION

	Seq.	Merge	Diverge	Cross
# OCCs	22	12	7	73
# Original Scenarios	697	152	8789	152
# Unique Original Intersections	176	80	405	80
# Unique OCC Intersections	9	10	5	10

not commonly present in the dataset. As a result, these types of scenarios are difficult to generate for us.

C. Intersection Variety

The ability to generate collisions on a variety of intersections is a main advancement of this work. Compared to some previous works which focused on automatic generation of OCCs on only one intersection [13], we show in Table IV that we can generate OCCs on a wide variety of intersections contained in the nuPlan dataset [3]. Across our small scale testing runs, we pulled in many starting scenarios (called “original” in the second row of Table IV). Some of these scenarios exist on the same intersections, and so we filter the initial number down to show the number of unique intersections pulled in from the dataset for attempted generation, shown in the third row. From there, the generation process pares down the scenarios until we are left with a smaller, but still fairly diverse group of intersections, shown in the fourth row. The cross OCC generator has the least intersection diversity of the group, with only 10 unique intersections out of 73 OCCs. This is in mainly due to some scenarios being particularly fertile ground for generating cross type OCCs. Partly because some scenarios have a large amount of valid spawn positions and speeds for conflict vehicles that can result in an OCC, and partly because cross scenarios in particular allow more lanes to potentially host both the conflict and the occluding vehicle.

D. Generation Efficiency

Generation efficiency is the measure of how easy it is for a method to generate the desired type of scenario. Ta-

ble V showcases the generation efficiencies for each of our 4 scenario generators, both when running in full and reduced modes, where reduced implies that all vehicles other than ego, the conflict vehicle and the occluder have been removed. Removing other vehicles in the scene has mixed results on the generation efficiency. It often results in more OCCs being generated from the same initial set of input scenarios, likely due to there being more free space on the road. In contrast, the number of OCCs generated compared to the number of generated scenarios does not show the same relationship. The number of generated scenarios per OCC is important since all generated scenarios need to be simulated to check them whether they result in an OCC, which requires time and computational resources. A method that generates large amounts of scenarios with a low probability of OCCs will need to spend more time running and checking those scenarios. For this reason, it is important to examine both measures of efficiency: OCCs per initial input scenarios, and OCCs per generated scenarios. The first one describes the limitations of a method with respect to how much data is available, and the second with respect to how much compute is available. In comparison to a previous work that only injected the occluder [13], which was only capable of generating 80 dynamic OCCs out of 105914 generated scenarios, our method generates far more OCCs per generated scenarios number of scenarios on even the most difficult type of conflict (diverge).

E. Severity and Direction of Impacts

The severity of an impact can be measured via the ΔV , or change in velocity, before and after the collision. The S0-3 severity scores outlined in [12] map onto the Abbreviated Injury Scale (AIS) [4] scores in Table VI via Table VII. Prior work [13] used the same methodology, and when comparing using the same ΔV intervals, we obtained similar distributions of S0-3 severity scores in the generated OCCs for cross collisions, which were the most common type that were generated in that work. However, the ΔV intervals used in [13] were incorrectly chosen. In our work, we have chosen to use a different set of intervals from [28], defined in Table VIII. It focuses on the more conservative of range that was available in order to generate an upper bound on OCC severity. Unlike [13], we do not assume the conflict vehicle that ego vehicle is colliding with is much heavier than ego to get our ΔV values. Instead, we assume a perfectly inelastic collision where both vehicles weigh roughly the same amount and calculate ΔV via Eq. (1), where the values are defined as follows: (m_A : Agent Mass, m_E : Ego Mass, v_A : Agent Speed, v_E : Ego Speed, θ : Heading Difference). Side collisions have lower bounds for severity because of the lower amount of crash structure between the occupants and the vehicles when compared to front or rear collisions.

$$\Delta V = \frac{m_A}{m_E + m_A} \sqrt{v_E^2 + v_A^2 - 2v_E v_A \cos(\theta)} \quad (1)$$

Collisions generated by the diverge and sequential collisions were the mildest because these collisions are all front to back as seen in Fig. 4, where the majority of the crash structure

TABLE V
GENERATION EFFICIENCY

	# Initial Scenarios	# Generated Scenarios	# Ego-to-veh. Collisions	# OCCs	# OCCs # Init'l Scens.	# OCCs # Gen'd Scens.
Reduced Sequential	697	52	26	22	3.16%	42.31%
Full Sequential	697	21	10	9	1.29%	42.86%
Reduced Cross	152	1465	111	73	48.03%	4.98%
Full Cross	152	634	42	20	13.16%	3.15%
Reduced Merge	152	875	34	12	7.89%	1.37%
Full Merge	152	475	20	10	6.58%	2.11%
Reduced Diverge	8789	200	21	7	0.08%	3.50%
Full Diverge	8789	159	25	9	0.10%	5.66%

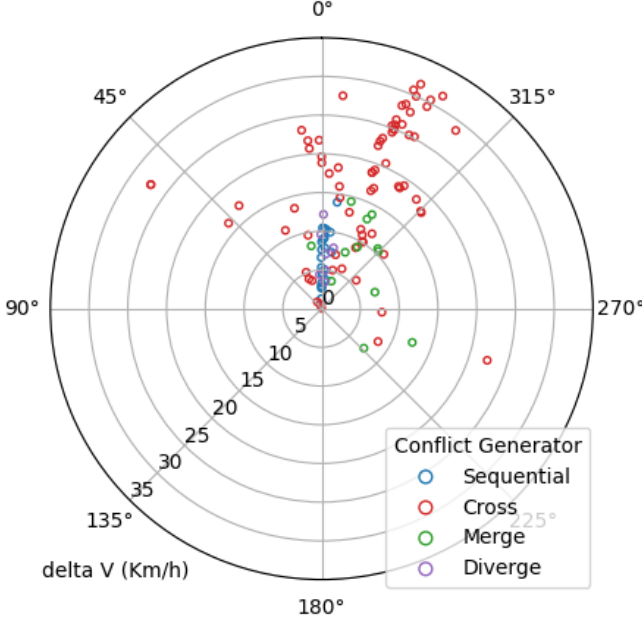


Fig. 4. Radial plot of impact angles and ΔV for each generator. Impact angles are relative to ego's heading when ego collides with another vehicle in an OCC.

can provide protection. They also largely happen with both vehicles moving in the same direction, just at different speeds, which significantly brings down the ΔV from the beginning to the end of the crash. Both cross and merge generators generate a much wider variety of collision angles, which allows vehicles to strike in T-bone fashion, as well as achieve greater velocities at the moment of impact. The result is that these generators produce the most severe collisions, as seen in Fig. 5. The collisions produced by the cross generator are the ones most likely to resemble the left turn tag-on type collisions generated in [13] and [30].

F. Reaction Time Ablations

Changes to reaction time have the potential to greatly increase or decrease the difficulty of generating OCCs. The chosen reaction time for all other experiments using our notice mechanism (Fig. 2) is an uncloak reaction time of 1.5 s and a notice threshold of 1 s. These values are chosen to be on the mid to lower end of human reaction time, since we are attempting to increase the likelihood of creating an OCC by

TABLE VI
THE AIS INJURY SCALE CATEGORIZES SEVERITY FROM 1 (MINOR) TO 6 (FATAL) [4]

AIS Code	Injury	Example
1	Minor	Superficial Laceration
2	Moderate	Fractured Sternum
3	Serious	Open Fracture of Humerus
4	Severe	Perforated Trachea
5	Critical	Ruptured Liver with Tissue Loss
6	Maximum	Total Severance of Aorta

TABLE VII
CLASSIFICATION OF DAMAGE BASED ON SAFETY-RELATED PROBABILITY[28]

Category	Description
S0	No safety-related damage
S1	> 10% probability of AIS 1–6 (and not S2 or S3)
S2	> 10% probability of AIS 3–6 (and not S3)
S3	> 10% probability of AIS 5–6

constraining the amount of time a human has to react to the vehicle being obstructed by the dynamic occlusion. This can simulate a human panicking, being distracted, or tired. However, the ego vehicle, as well as any of the other vehicles on the road may be driven by an autonomous system, which may be capable of reacting more quickly. In order to test whether our generation methods are robust to more reactive agents, we lower both the uncloak reaction time and the notice threshold in our generated OCCs.

1) *Ego Reaction Time*: In Table IX, we gradually improve ego's reaction time while maintaining the reaction time of

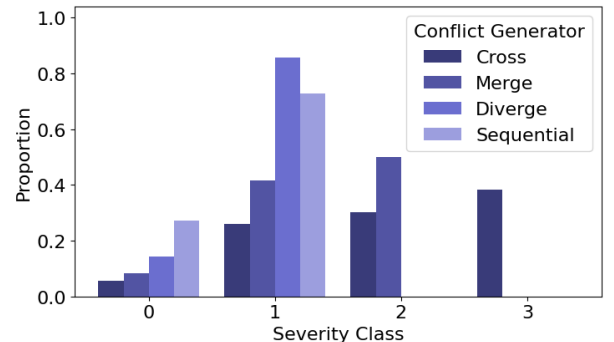


Fig. 5. Severity class distributions of different types of OCC

TABLE VIII
COLLISION SEVERITY CATEGORIZATION BY IMPACT SIDE AND ΔV
BASED ON CONSERVATIVE INTERVALS FROM [28]

Collision Side	ΔV Range km/h	Category
Front/Back	$\Delta V \leq 4$	0
	$4 < \Delta V \leq 20$	1
	$20 < \Delta V \leq 40$	2
	$40 < \Delta V$	3
Side	$\Delta V \leq 2$	0
	$2 < \Delta V \leq 8$	1
	$8 < \Delta V \leq 16$	2
	$16 < \Delta V$	3

TABLE IX
PERCENT OF OCCs STILL OCCURRING AFTER VARYING EGO REACTION
DELAY (NON-EGO REACTIONS SET TO (1.5, 1.0))

#OCCs %	(unlock_reaction_time, notice_threshold)			
	(1.5 s, 1.0 s)	(1.0 s, 0.67 s)	(0.5 s, 0.33 s)	
Reduced Sequential	22 100%	12 54.55%	7 31.82%	
Reduced Cross	73 100%	59 80.82%	47 64.38%	
Reduced Merge	12 100%	9 75.00%	7 58.33%	
Reduced Diverge	7 100%	2 28.57%	1 14.29%	

TABLE X
PERCENT OF OCCs STILL OCCURRING AFTER VARYING AGENT
REACTION DELAY (EGO REACTIONS SET TO (0.5, 0.33))

#OCCs %	(unlock_reaction_time, notice_threshold)			
	(1.5 s, 1.0 s)	(1.0 s, 0.67 s)	(0.5 s, 0.33 s)	
Reduced Sequential	7 100%	7 100%	8 114.3%	
Reduced Cross	47 100%	28 59.57%	22 46.81%	
Reduced Merge	7 100%	3 42.86%	1 14.28%	
Reduced Diverge	1 100%	1 100%	1 100%	

all non-ego agents, while measuring the absolute number and percentage of previously generated OCCs that still result in a collision. At ego's reaction time improves, simulating a world where ego is an ADS and everyone else is a human, we observe that some OCCs generated by each generator would not have occurred. Cross type OCCs are the most robust with respect to the improved reaction time and notice threshold.

2) *All Vehicle Reaction Time*: In Table X, we start with the ego's reaction time being lowered to the final rightmost column in Table IX, and gradually improve all other agent's reaction times. This simulates a situation where all vehicles are driven by ADS, or by very attentive drivers. We see a similar result, with fewer OCCs still occurring in most categories. Sequential and diverge OCCs are largely unchanged due to the fact that the conflict vehicle starts stationary in all sequential and diverge scenarios ahead of both other vehicles. This means that the vehicles have little agency in the collision. In the final column, we actually observe an increase in the number of OCCs at the fastest reaction time. This can result if the extra information being visible earlier resulted in a slower acceleration of the initially stopped vehicle. In the cross and merge type scenarios, we see the same type of dropoff as observed in the previous table.

V. RELATED WORK

Related research can be split into two intersecting categories: ADS validation, and occlusion related planning. In the former, the focus is on optimizing driver behaviour in each scenario in order to attempt to create realistic collisions in an adversarial way, such as with STRIVE [26]. The latter consider reachability constraints of potential vehicles in occluded regions [14, 24, 32] and attempt to find ways to guarantee safe navigation of these urban roadways [33]. At the intersection of these two categories are papers that specifically focus on scenario generation in the occluded regime. Agarkar et al. [1] focuses primarily on generation of large numbers of scenarios via fuzzing of a base scenario. It modifies weather and lighting conditions that complicate the perception task and injects static occlusions that block pedestrians to create its scenarios. Muslim et al. [21] is based exclusively on the cut-out scenario on the highway, which we also explore, though in the context of a diverging lane rather than a lane change. They propose a data driven technique for validating ADSs in this particular scenario, though do not actually perform the experiments. AdvSim [31] takes a different approach to ours by converting existing scenarios to be run in a LiDAR simulator and extrapolating what occluded areas would look like, so that they can simulate LiDAR in the occluded regime. Rather than injecting new agents, they modify the behaviour of existing agents to cause collisions in their simulation. As mentioned previously, the closest prior work [13] performed similar agent injections to create OCCs. Their technique is built to work on only a single intersection. Unlike ours, they only inject the occluder and focus on creating conflicting trajectories via satisfying a Nash equilibrium in an occluded regime, rather than relying on existing planners. The DriveOcclusionSim benchmark in [30] generates its scenarios by randomly placing vehicles, obstacles and VRUs in the simulation. It focuses on slightly different types of occluded scenarios than us including VRUs emerging from behind parked cars, red light infractions, and the canonical left turn occlusion covered in both our work and [13].

VI. LIMITATIONS AND EXTENSIONS

In this section, we acknowledge a number of limitations and mention some as-yet unexplored extensions.

As seen in the second part of our reaction time ablations, in rare occasions some scenarios generated as OCCs may be the result of a weak planner reacting unpredictably to the on-road conditions rather than anything to do with the occlusion. Another limitation is the lack of direct traffic light data in the nuPlan dataset. The nuPlan simulator infers traffic light status on a per-lane basis based on recorded agent movement, which results in empty intersections always being green, and any lane with only a single stopped agent in the scene evaluating to red, even if the ground truth is different, which can result in spawn points that are less realistic than normal.

In the creation of this work, we identified several paths that we had not fully explored that may merit future exploration. We did not attempt to generate dynamic occlusions where the occluded agent is a pedestrian or cyclist, in part because

pedestrians and all other non vehicle tracks are not controlled in a reactive manner in nuPlan, but instead simply part of recorded playback. This can result in pedestrians and cyclists making poor decisions when crossing the road and walking into stopped vehicles. We did not explore varying the size or type of the inserted vehicles, due to the already substantial combinatorial explosion of potential OCCs that would be generated. We also did not explore slight perturbations in the inserted positions of both the conflict vehicle, and the occluder despite the fact that it would make some OCCs possible where they had not been before for the same reason. Multiple different planners are available to control the ego and other agents in our experiments. Experimenting with what kind of OCCs can be generated with different planners and how choosing different planners for different vehicles would influence the results should be explored in future work. In small scale experiments, it was found, however, that generating OCCs for significantly weaker planners was less reliable, since weak planners would often move unpredictably and create OCCs simply because of poor decisions in the occluded regime that had nothing to do with the occlusion itself. The nuPlan simulator [3] along with our modifications to it to turn it into OCC-Gen is a capable platform for exploring these remaining items in future work.

VII. CONCLUSION

We have introduced a method to generate dynamic occlusion scenarios crucial for safety validation in autonomous driving systems. Our approach demonstrates effective scenario generation across diverse road geometries and driver behaviours, addressing the scarcity of critical occlusion scenarios in existing datasets. By leveraging our custom simulation modifications and scenario generation techniques, we successfully created a variety of occlusion-induced collision scenarios, enhancing the robustness of validation efforts. This method broadens the scope of safety testing and contributes to advancing the safety of autonomous driving systems.

REFERENCES

- [1] A. S. Agarkar, R. Gandhiraj *et al.*, “Simulation based closed loop scenario fuzzing for safety critical ADAS applications,” in *2023 IEEE 20th India Council International Conference (INDICON)*, Dec. 2023, pp. 759–764.
- [2] N. L. Broen and D. P. Chiang, “Braking response times for 100 drivers in the avoidance of an unexpected obstacle as measured in a driving simulator,” *Proceedings of the Human Factors and Ergonomics Society Annual Meeting*, vol. 40, no. 18, pp. 900–904, 1996.
- [3] H. Caesar, J. Kabzan *et al.*, “NuPlan: A closed-loop ML-based planning benchmark for autonomous vehicles,” Feb. 2022, arXiv: 2106.11810.
- [4] W. S. Copes, H. R. Champion *et al.*, “Progress in characterizing anatomic injury,” *Journal of Trauma and Acute Care Surgery*, vol. 30, no. 10, pp. 1200–1207, 1990.
- [5] K. Czarnecki, “Operational world model ontology for automated driving systems - Part 1: Road structure,” Waterloo Intelligent Systems Engineering (WISE) Lab, University of Waterloo, Tech. Rep., 2018.
- [6] —, “Operational world model ontology for automated driving systems - Part 2: Road users, animals, other obstacles, and environmental conditions,” Waterloo Intelligent Systems Engineering (WISE) Lab, University of Waterloo, Tech. Rep., 2018.
- [7] D. Dauner, M. Hallgarten *et al.*, “Parting with misconceptions about learning-based vehicle motion planning,” 2023, arXiv: 2306.07962.
- [8] J. C. F. de Winter, M. Saffarian, and J. W. Senders, “The effect of an occlusion-induced delay on braking behavior in critical situations: A driving simulator study,” *Human Factors*, vol. 65, no. 7, pp. 1336–1344, 2023.
- [9] P. Drożdżel, S. Tarkowski *et al.*, “Drivers’ reaction time research in the conditions in the real traffic,” *Open Engineering*, vol. 10, no. 1, pp. 35–47, 2020.
- [10] J. Engström, S.-Y. Liu *et al.*, “Modeling road user response timing in naturalistic traffic conflicts: a surprise-based framework,” *Accident Analysis & Prevention*, vol. 198, p. 107460, 2024.
- [11] M. Hallgarten, M. Stoll, and A. Zell, “From prediction to planning with goal conditioned lane graph traversals,” 2023, arXiv: 2302.07753.
- [12] ISO Central Secretary, “Road vehicles — functional safety — part 3: Concept phase,” International Organization for Standardization, Geneva, CH, Standard ISO TR 26262-3:2018, 2018.
- [13] M. Kahn, A. Sarkar, and K. Czarnecki, “I know you can’t see me: Dynamic occlusion-aware safety validation of strategic planners for autonomous vehicles using hypergames,” in *2022 International Conference on Robotics and Automation (ICRA)*, May 2022, pp. 11 202–11 208.
- [14] M. Koschi and M. Althoff, “Set-based prediction of traffic participants considering occlusions and traffic rules,” *IEEE Transactions on Intelligent Vehicles*, vol. 6, no. 2, pp. 249–265, 2020.
- [15] K. D. Kusano, J. M. Scanlon *et al.*, “Framework for a conflict typology including contributing factors for use in ads safety evaluation,” in *27th International Technical Conference on the Enhanced Safety of Vehicles (ESV) National Highway Traffic Safety Administration*, no. 23-0328, 2023.
- [16] A. Laureshyn and A. Várhelyi, “The Swedish traffic conflict technique: Observer’s manual,” Lund University, Tech. Rep., 2018.
- [17] N. D. Lerner, “Brake perception-reaction times of older and younger drivers,” *Proceedings of the Human Factors and Ergonomics Society Annual Meeting*, vol. 37, no. 2, pp. 206–210, 1993.
- [18] G. Markkula, “Modeling driver control behavior in both routine and near-accident driving,” in *Proceedings of the human factors and ergonomics society annual meeting*, vol. 58, no. 1. SAGE Publications Sage CA: Los Angeles, CA, 2014, pp. 879–883.
- [19] Ministry of Transportation, “Geometric design standards for Ontario highways,” 1985, Survey and Design Office, Ontario, Canada.

- [20] N. Montali, J. Lambert *et al.*, “The Waymo open sim agents challenge,” 2023, arXiv: 2305.12032.
- [21] H. Muslim, S. Endo *et al.*, “Cut-out scenario generation with reasonability foreseeable parameter range from real highway dataset for autonomous vehicle assessment,” *IEEE Access*, vol. 11, pp. 45 349–45 363, 2023.
- [22] N. H. T. S. A. (NHTSA), “National motor vehicle crash causation survey: Report to congress,” p. 059, 2008.
- [23] E. J. Nilsson, M. L. Aust *et al.*, “Effects of cognitive load on response time in an unexpected lead vehicle braking scenario and the detection response task (drt),” *Transportation Research Part F: Traffic Psychology and Behaviour*, vol. 59, pp. 463–474, 2018.
- [24] P. F. Orzechowski, A. Meyer, and M. Lauer, “Tackling occlusions & limited sensor range with set-based safety verification,” in *2018 21st International Conference on Intelligent Transportation Systems (ITSC)*. IEEE, 2018, pp. 1729–1736.
- [25] E. J. Ploran, S. M. Nelson *et al.*, “Evidence accumulation and the moment of recognition: dissociating perceptual recognition processes using fmri,” *Journal of Neuroscience*, vol. 27, no. 44, pp. 11 912–11 924, 2007.
- [26] D. Rempe, J. Pailon *et al.*, “Generating useful accident-prone driving scenarios via a learned traffic prior,” 2022, arXiv: 2112.05077.
- [27] R. Roriz, J. Cabral, and T. Gomes, “Automotive lidar technology: A survey,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, no. 7, pp. 6282–6297, 2021.
- [28] *Considerations for ISO 26262 ASIL Hazard Classification*, SAE, 2018, j2980_201804.
- [29] O. Scheel, L. Bergamini *et al.*, “Urban driver: Learning to drive from real-world demonstrations using policy gradients,” 2021, arXiv: 2109.13333.
- [30] H. Shao, L. Wang *et al.*, “ReasonNet: End-to-end driving with temporal and global reasoning,” 2023, arXiv: 2305.10507.
- [31] J. Wang, A. Pun *et al.*, “AdvSim: Generating safety-critical scenarios for self-driving vehicles,” in *2021 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, Jun. 2021, pp. 9904–9913.
- [32] M.-Y. Yu, R. Vasudevan, and M. Johnson-Roberson, “Occlusion-aware risk assessment for autonomous driving in urban environments,” *IEEE Robotics and Automation Letters*, vol. 4, no. 2, pp. 2235–2241, 2019.
- [33] Z. Zhang and J. Fisac, “Safe occlusion-aware autonomous driving via game-theoretic active perception,” in *Robotics: Science and Systems XVII*, ser. RSS2021. Robotics: Science and Systems Foundation, Jul. 2021.



Eli-Henry Dykhne is a MMath student in the department of Computer Science at the University of Waterloo, Canada. He works in the Waterloo Autonomous Systems Lab under Professor Krzysztof Czarnecki on safety verification for prediction and planning systems. He obtained his bachelors degree at the University of Guelph, Canada in 2022. His research interests include driving simulation, safety verification of autonomous systems, reinforcement learning, and applications of machine learning techniques to autonomous driving.



Shea Anthony Cardozo is a PhD student in the David R. Cheriton School of Computer Science at the University of Waterloo. He works in the Waterloo Autonomous Systems Lab under Professor Krzysztof Czarnecki on deep learning-based planning systems for autonomous vehicles. He received an undergraduate degree in Statistics from the University of Waterloo in 2021 and a Masters degree in Computer Science from the University of Toronto in 2022.

Previously, Shea worked in industry as a Machine Learning Scientist at the computer vision startup Tenyks (YC'S21) from 2022-2023, researching and prototyping new explainability and interpretability methods for classification, detection, and segmentation tasks. He has additionally held positions as a data scientist and actuarial analyst at Noom, Intact Insurance, Moodys Analytics and the Royal Bank of Canada. In addition to his studies, he currently works as a teaching and instructor assistant for multiple courses at the University of Waterloo.

Shea is affiliated with the Vector Institute and is a member of the Computer Vision Foundation. He has received numerous awards and accolades for his work, such as the University of Waterloo Graduate Scholarship and the David R. Cheriton Graduate Scholarship.



James Riddell received a B.A. degree in Mathematics from The University of British Columbia, Vancouver, BC, Canada in 2022. He is currently working toward a MMath in computer science at the University of Waterloo. His primary research interests include generative AI, foundation models, safety critical scenario generation, and scenario understanding within the driving domain.

Sean Sedwards is a Research Assistant Professor with the Department of Electrical and Computer Engineering, University of Waterloo, Canada. He was formerly Ingénieur Expert de Recherche with the Institut National de Recherche en Informatique et en Automatique Rennes – Bretagne, France. He received his PhD from the Microsoft Research, University of Trento Center for Computational and Systems Biology, Italy. His research interests include scalable formal verification and the safety and robustness of machine learning applied to autonomous driving.



Krzysztof Czarnecki co-lead the development of the first ADS tested on public roads in Canada in 2018. He is currently a Professor of electrical and computer engineering and a University Research Chair with the University of Waterloo, where he heads the Waterloo Intelligent Systems Engineering (WISE) Laboratory. He is a Leading Expert in the safety of automated driving systems (ADS), with a focus on assuring the safety of driving behavior and machine-learned functions. As a member of standardization committees, he has contributed to

ISO 21448 (second edition), ISO 8800 (under development), and SAE J3164. He received the Premier's Research Excellence Award in 2004 and the British Computing Society in Upper Canada Award for Outstanding Contributions to IT Industry in 2008. He has also received eight best paper awards, two ACM Distinguished Paper Awards, and four most influential paper awards.